Server Deployment & Camera HTTPS Configuration Guide

Document Version: 1.0 | **Topic:** WebRTC Camera Security, SSL/TLS, Nginx Reverse Proxy, FastAPI & Vite Deployment | **Status:** Production Ready

CORE ANSWER SUMMARY:
Will the selfie camera work on a server?
YES, ABSOLUTELY — but **ONLY if the server is served over HTTPS (SSL/TLS)**.
All modern browsers (Chrome, Safari, iOS Safari, Firefox, Edge, Android) enforce the **W3C Secure Context Standard**. The navigator.mediaDevices.getUserMedia camera API is strictly disabled on insecure HTTP connections over network IPs (e.g. http://192.168.x.x or http://your-server-ip). The only origin where HTTP works is localhost (127.0.0.1) for local testing. To allow hotel guests to use the live front selfie camera from their phones, you must configure a domain with an SSL certificate or an HTTPS tunnel.

1. Understanding the Web Browser Security Model

Modern web browsers categorize web origins into **Secure Contexts** and **Insecure Contexts** to protect user privacy:

Origin / Protocol	Context Type	Camera API (getUserMedia)	Use Case
http://localhost:5173	Secure Context (Exemption)	**ALLOWED**	Local development machine
http://127.0.0.1:5173	Secure Context (Exemption)	**ALLOWED**	Local loopback testing
http://192.168.1.50:5173	Insecure (LAN IP over HTTP)	**BLOCKED by Browser**	Opening on phone via Wi-Fi IP
http://hotel.example.com	Insecure (Plain HTTP)	**BLOCKED by Browser**	Server without SSL
https://hotel.example.com	**Secure Context**	**FULL ACCESS (Allowed)**	**Production Server with SSL**
https://*.trycloudflare.com	**Secure Context**	**FULL ACCESS (Allowed)**	Instant remote testing/staging

2. Required Software & Tools to Install on the Server

To deploy this full-stack system (FastAPI backend + Vite React frontend + MySQL database) with HTTPS camera support on a cloud Linux server (Ubuntu 22.04 / 24.04 LTS), install the following packages:

Software / Tool	Purpose in Hotel System	Installation Command (Ubuntu/Debian)
Nginx	Web Server & Reverse Proxy. Serves static React files and routes /api/ to FastAPI.	sudo apt install -y nginx
Certbot & Plugin	Obtains free, automated SSL/TLS certificates from Let's Encrypt with auto-renewal.	sudo apt install -y certbot python3-certbot-nginx

Node.js (v18+) & npm	Builds production frontend bundle (npm run build producing the dist/ folder).	curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt install -y nodejs
Python 3.10+ & pip	Executes FastAPI backend, SQLAlchemy, and Uvicorn.	sudo apt install -y python3 python3-pip python3-venv
MySQL Server	Database storing rooms, bookings, guests, selfies, invoices, and audit logs.	sudo apt install -y mysql-server
systemd (Built-in)	Keeps FastAPI running continuously in the background and auto-restarts on server reboot.	Built into Linux OS

3. Step-by-Step Production Setup Instructions

Follow these 5 straightforward steps on your production server:

Step 1: Point Your Domain DNS to the Server

• Go to your domain registrar (GoDaddy, Namecheap, Cloudflare, AWS Route53).
• Create a **DNS 'A' Record**:
Host: hotel.yourdomain.com (or @ for root domain)
Value / IP: [Your Server's Public IPv4 Address]

Step 2: Build the Frontend for Production

In the frontend folder, build the optimized static assets:

```
# 1. Navigate to frontend folder
cd /var/www/hotel_reservation/frontend
# 2. Install dependencies & build production bundle
npm install
npm run build
# This generates the /dist directory with optimized HTML/JS/CSS
```

Step 3: Configure FastAPI Background Service (systemd)

Create a systemd service file at /etc/systemd/system/hotel-backend.service:

```
[Unit]
Description=Hotel Reservation FastAPI Backend
After=network.target mysql.service

[Service]
User=www-data
WorkingDirectory=/var/www/hotel_reservation/backend
ExecStart=/var/www/hotel_reservation/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port
8000 --workers 4
Restart=always
RestartSec=5
Environment="DATABASE_URL=mysql+pymysql://hotel_user:SecurePass123@localhost/hotel_reservation"

[Install]
WantedBy=multi-user.target
```

Enable and start the service:
sudo systemctl daemon-reload && sudo systemctl enable hotel-backend && sudo systemctl start hotel-backend

Step 4: Configure Nginx as Reverse Proxy & Static Host

Create configuration file /etc/nginx/sites-available/hotel.conf:

```
server {
server_name hotel.yourdomain.com;

# Static React frontend root
root /var/www/hotel_reservation/frontend/dist;
index index.html;

# SPA routing: all page paths redirect to index.html
location / {
try_files $uri $uri/ /index.html;
}

# Proxy API calls directly to FastAPI on port 8000
location /api/ {
proxy_pass http://127.0.0.1:8000;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection 'upgrade';
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

Activate configuration: sudo ln -s /etc/nginx/sites-available/hotel.conf /etc/nginx/sites-enabled/ && sudo nginx -t && sudo systemctl reload nginx

Step 5: Obtain Free SSL Certificate via Certbot (Enables Camera)

Run the automated Certbot Nginx installer:

```
sudo certbot --nginx -d hotel.yourdomain.com
```

Certbot automatically provisions the Let's Encrypt SSL certificate, updates Nginx to listen on port 443 with TLS 1.3, configures automatic HTTP-to-HTTPS redirects, and sets up automatic cert renewal.

4. Quick HTTPS Testing for Mobile Phones (Without Buying a Domain)

If you want to immediately test the live front-facing mobile selfie camera from an iPhone or Android phone on your current machine or server without purchasing a domain, use either of these instant HTTPS tunneling methods:

Method A: Cloudflare Tunnel (Free, Instant, No Sign-up Required)

1. Download cloudflared on your machine or server.
2. Run: cloudflared tunnel --url http://localhost:5173
3. Cloudflare outputs a public HTTPS link (e.g. https://alpha-hotel-test.trycloudflare.com).
4. Open that URL on your phone — the camera will open immediately with full hardware permission!

Method B: ngrok Tunnel

1. Run: ngrok http 5173
2. Open the resulting https://*.ngrok-free.app URL on any smartphone.
3. WebRTC camera permissions are 100% granted because the tunnel provides a valid SSL certificate.

5. Production Troubleshooting & Verification Matrix

Symptom / Error	Underlying Cause	Exact Resolution
Camera access unavailable or permission denied	The user accessed the site over plain HTTP (e.g., http://192.168.1.100).	Ensure user visits https://.... Configure Nginx to force redirect HTTP to HTTPS.
navigator.mediaDevices is undefined	Browser strictly blocked WebRTC APIs due to non-secure origin.	Switch to HTTPS or localhost. The frontend already detects this and offers the photo upload fallback.
NotAllowedError / Permission denied	User clicked 'Block' or 'Don't Allow' on browser permission pop-up.	Tap the lock icon in the browser address bar -> Permissions -> Camera -> Allow.
Black screen on camera preview	Missing playsInline, autoPlay, or muted attributes on video element.	Already implemented and solved in GuestPortal.jsx.
QR code points to wrong address	QR code URL hardcoded to localhost instead of production domain.	Set frontend environment variable VITE_PUBLIC_URL=https://hotel.yourdomain.com.

6. Summary & Recommendations

• **Code is 100% production-ready:** Your existing React frontend camera logic (facingMode: 'user', auto-retake, confirm, canvas snapshot, and upload fallback) requires **zero code changes**.
• **Only server-level SSL is needed:** Installing **Nginx** and running **Certbot** on your server takes less than 5 minutes and is completely free.
• **Resilient Fallback:** Even if a guest's browser has strict camera restrictions or hardware damage, the integrated **Upload Selfie Fallback** ensures 100% check-in completion without friction.